

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 1 — Lo stesso prompt su più modelli Claude

Prima di cominciare

I materiali del corso

Le dispense e il repository delle esercitazioni, per tutto il corso

- **Le dispense in PDF** di ogni parte di ogni modulo, teoria e pratica, in due posti: l'ultima release, `https://github.com/simonescannapieco/claude-code-hands-on-dgroove-tecnodata-trentina/releases/latest`, e la cartella `dispense` del repository, divisa in `theory` e `practice`: `https://github.com/simonescannapieco/claude-code-hands-on-dgroove-tecnodata-trentina/tree/master/dispense`
- **Il repository delle esercitazioni**: tutte le demo e gli esercizi — `https://github.com/simonescannapieco/claude-code-hands-on-dgroove-tecnodata-trentina`; si clona una volta, poi si aggiorna con `git pull`:

```
git clone https://github.com/simonescannapieco/\
claude-code-hands-on-dgroove-tecnodata-trentina esercitazioni
```

Le dispense | Il repository

Prima di cominciare

Quattro controlli dell’installazione

```
# nel terminale Linux o WSL 2, nella cartella esercitazioni
git pull                        # gli elementi del modulo
claude --version                # 2.1.293 o successiva
tools/verifica_ambiente.sh      # deve finire con: Tutto pronto
claude doctor                   # se qualcosa non torna
```

- `claude --version`: **2.1.293 o successiva** — con le versioni precedenti l’alias `haiku` punta a un modello più vecchio
- In Claude Code, `/status`: il metodo di accesso attivo dev’essere il vostro **abbonamento Pro**
- **Nessuna** `ANTHROPIC_API_KEY`: se c’è, Claude Code la usa al posto dell’abbonamento — e con `claude -p`, che userete nell’esercitazione, sempre; l’uso dell’API non è incluso nel piano Pro
- `tools/verifica_ambiente.sh` **tutto verde**: controlla anche la chiave, senza mai stamparla; `claude doctor` è una diagnostica in sola lettura

Il ciclo di ogni esercizio

Preparare, lavorare in Claude Code, verificare — e, se serve, ricominciare

```
# dalla cartella esercitazioni
tools/prepara_esercizio.sh M1-E01   # lo spazio di lavoro
cd ~/corso-claude-code/M1-E01       # fuori dal repository
claude                              # i passi del README
# poi /exit, e si torna nella cartella esercitazioni:
cd -
tools/verifica.sh M1-E01            # 0 = fatto, 1 = non ancora
tools/reset_esercizio.sh M1-E01     # da capo, se serve
```

1. **Prepara**: crea `~/corso-claude-code/M1-E01/`, fuori dal repository, con il punto di partenza; è un repository git con un primo commit, e `git status` e `git diff` mostrano che cosa avete cambiato
2. **Lavora**: `claude` nello spazio di lavoro, con i «Passi in Claude Code» del `README.md` dell'elemento
3. **Verifica**: esce con 0 se il risultato è quello atteso, con 1 se non ancora (e dice che cosa manca), con 2 se qualcosa impedisce la verifica
4. **Ricomincia**: cancella lo spazio di lavoro, dopo aver chiesto conferma, e lo prepara di nuovo

Un ambiente verificato è metà dell'esercizio.

Il piano Pro, con misura

Cinque abitudini, per tutte le sessioni

- `/usage`: le barre dei limiti del piano — un limite di sessione che si azzera ogni cinque ore e uno settimanale, **condivisi** fra claude.ai e Claude Code
- **Sonnet per il lavoro di routine**: Opus consuma sensibilmente di più il limite d'uso; si cambia con `/model`
- **Effort basso quando basta**: è più rapido ed economico; si regola con `/effort`
- `/clear` **fra un esercizio e l'altro**: ripartire da zero non costa nulla
- **Niente Fable**: sul piano Pro funziona solo con i crediti d'uso (*usage credits*), a pagamento — le esercitazioni non ne hanno bisogno